

多項式・形式的べき級数 線形代数 固有値・固有ベクトル

固有関数による dp 高速化

1. 概要

本講座では、ある種の dp を高速化するテクニックについて解説します。

線形代数を学んだことがある方は、固有値・固有ベクトルあるいは行列の対角化によって行列の M 乗計算を高速化する手法を見たことがあるかもしれません。本講座の内容は、同様のアイデアで dp 遷移の反復を高速化するものです。特に、形式的べき級数の微分を含む dp 遷移が本講座の主な対象になります。

2. 前提知識

線形代数（行列とベクトル）。

多項式・形式的べき級数，合成アルゴリズムの特殊ケース。

変数分離形の微分方程式。

3. 固有値・固有ベクトル

3.1. 定義

【定義 1】

V をベクトル空間， $f: V \longrightarrow V$ を線形写像とします。 $v \in V \setminus \{0\}$ および定数 λ が

$$f(v) = \lambda v$$

を満たすとき、 λ を**固有値** (eigenvalue), v を λ に関する**固有ベクトル** (eigenvector) という.

V の要素として関数や多項式を考えている場合には、固有ベクトルのことを**固有関数** (eigenfunction) ともいう.

注意

ベクトル空間や線形写像について知らない方は,

- V は列ベクトル全体 ($N \times 1$ 行列全体).
- f は, A を $N \times N$ 行列として $f(x) = Ax$ により定義されるもの.

と考えてもらえればほぼ問題ありません. ただし本講座内の解説では, 形式的べき級数全体を V とするなど, (有限の大きさの) 行列やベクトルの形では扱えないものも考えます.

3.2. 例

- $A = \begin{pmatrix} 1 & -2 \\ 1 & -2 \end{pmatrix}$ とし, 列ベクトル x に対して $f(x) = Ax$ とするとき,
 - $\begin{pmatrix} 2 \\ 1 \end{pmatrix}$ は f の固有値 0 に関する固有ベクトルである.
 - $\begin{pmatrix} 4 \\ 2 \end{pmatrix}$ は f の固有値 0 に関する固有ベクトルである.
 - $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ は f の固有値 -1 に関する固有ベクトルである.
- V を形式的べき級数全体とし, $f: V \rightarrow V$ を形式的べき級数の微分とすると, 定数 λ に対して, $e^{\lambda x}$ は固有値 λ に関する固有関数である.

3.3. 行列累乗への利用

v_k が固有値 λ_k に関する固有ベクトルであるとき, $(A^M)v_k = \lambda_k^M v_k$ が成り立ちます.

したがって, 固有値・固有ベクトルの組 (λ_k, v_k) ($0 \leq k \leq n$) が見つかっており, ベクトル v が $v = \sum_{k=0}^n c_k v_k$ と固有ベクトルの線形結合で書けると,

$$(A^M)v = \sum_{k=0}^n (c_k \lambda_k^M) v_k$$

によって、 $(A^M)v$ を求めることができます。

この考え方を dp 高速化に利用することが本講座の主要なテーマです。

4. 問題

本講座では、テクニックそのものを直接説明するのではなく、いくつかの問題の解説を通して紹介していきます。

4.1. 問題 2

【問題 2】

数列 $(A_0, A_1, \dots, A_N)$ が与えられます。次によって定まる dp テーブル $\text{dp}[i][j]$ ($0 \leq i \leq M, 0 \leq j \leq N$) を考えます。

- $\text{dp}[0][j] = A_j$ ($0 \leq j \leq N$)
- $\text{dp}[i+1][j] = j \cdot \text{dp}[i][j] + (j+1) \cdot \text{dp}[i][j+1]$.
ただし $j = N$ の場合には $\text{dp}[i][j+1] = 0$ として扱う。

$\text{dp}[M][0], \dots, \text{dp}[M][N]$ を 998244353 で割った余りを出力してください。

この問題は $O(N(\log N + \log M))$ 時間で解くことができます。

まずは、多項式を用いて言い換えましょう。

【問題 2 の言い換え】

N 次以下の多項式 $A_0(x)$ が与えられます。 $A_1(x), A_2(x), \dots, A_M(x)$ を次によって定義します：

$$A_{i+1}(x) = (1+x)A'_i(x).$$

$A_M(x)$ を mod 998244353 で出力してください.

線形代数の言葉で言い換えると次のような問題です.

- V を N 次以下の多項式全体とする.
- $f: V \rightarrow V$ を $f(A) = (1+x)A'$ により定める.
- $f^M(A)$ を求めよ.

この問題を固有関数を使って解きます.

固有値・固有関数がどのようなものかを考えましょう. $(1+x)A'(x) = \lambda A(x)$ が成り立つような定数 λ と多項式 $A(\neq 0)$ はどのようなものでしょうか.

これは**微分方程式**に関する問題です. この微分方程式は変数分離形という形で, 微分方程式を解くことで, 0 以上 N 以下の整数 λ に対して $A(x) = (1+x)^\lambda$ が固有値 λ の固有関数になっていることが分かります.

▶ 補足

見つかった固有関数および固有値から, この問題は次の手順で解けることが分かります.

- $A(x) = \sum_{k=0}^N c_k(1+x)^k$ であるような係数列 c_k を求める.
- c_k に k^M をかける.
- $\sum_{k=0}^N c_k(1+x)^k$ を出力する.

これは多項式の Taylor Shift を用いて $O(N(\log N + \log M))$ 時間で計算できます.

4.2. 問題 3

【問題 3】

N 次以下の多項式 $A_0(x)$ が与えられます. $A_1(x), A_2(x), \dots, A_M(x)$ を次によって定義します:

$$A_{i+1}(x) = A_i(x) + A'_i(x).$$

$A_M(x)$ を mod 998244353 で出力してください.

この問題は全体で $O((N + M) \log^2(N + M))$ 時間で解くことができます.

固有関数を求めるのは簡単で, $A(x) = e^{kx}$ が固有値 $(k + 1)$ の固有関数になります.

しかし $e^x \bmod x^{N+1}$ は固有関数にならないことなどが厄介です. 例えば $A(x) \equiv \sum_{k=0}^N c_k e^{kx} \pmod{x^{N+1}}$ となるような係数列 c_k を求めることは可能ですが, $N + 1$ 次以上の項が影響してしまうため, $A(x)$ を $\sum_{k=0}^N c_k e^{kx}$ に置き換えてしまうと正しく答を求めることができません.

そこで, 次のようにします.

- $A(x) \equiv \sum_{k=0}^{N+M} c_k e^{kx} \pmod{x^{N+M+1}}$ となる係数列 c_k を求める.
- c_k に $(k + 1)^M$ をかける.
- $\left(\sum_{k=0}^{N+M} c_k e^{kx} \right) \bmod x^{N+1}$ を出力する.

$N + M + 1$ 次以上の項が最終的には出力すべき多項式に影響しないため, $A(x) = \sum_{k=0}^{N+M} c_k e^{kx}$ であるとして問題を解けばよいというのが重要なポイントです.

最初の係数列 c_k を求めるステップでは, $A(x) \equiv C(e^x) \pmod{x^{N+M+1}}$ となる多項式 C を求めることが必要です. 一度 $A(x) \equiv C(1 - e^x) \pmod{x^{N+M+1}}$ となる形式的べき級数 $C(x)$ を求めてから, その $N + M$ 次以下の部分を残すという要領で計算できます. e^x 特有の合成アルゴリズムもありますし, 一般の形式的べき級数合成アルゴリズムを使うこともできます. 最後のステップも同様です.

この方法によりこの問題は全体で $O((N + M) \log^2(N + M))$ 時間で解くことができます.

4.3. 問題 4

【問題 4】

N 次以下の多項式 $A_0(x)$ が与えられます. さらに数列 $b_0, \dots, b_{M-1}$ も与えられます. $A_1(x), A_2(x), \dots, A_M(x)$ を次によって定義します:

$$A_{i+1}(x) = x(A_i(x) + A'_i(x)) + b_i A_i(x).$$

$A_M(x)$ を $\text{mod } 998244353$ で出力してください.

この問題は, $O((N + M) \log^2(N + M))$ 時間で解くことができます. なお, この漸化式は, [ABC272Ex](#) の解法のひとつで現れるものです.

ほとんどこれまでの問題と同様なので, 概要のみ述べます. まず $A(x) \mapsto x(A(x) + A'(x))$ の固有関数を求めると, $x^k e^{-x}$ が固有値 k の固有関数になることが分かります. $b_i A_i(x)$ を加算することも必要ですが, これはこの固有関数が $(k + b_i)$ 倍されるようになるという違いでしかありません.

そこで, 次のようにしてこの問題を解くことができます.

- $A(x) \equiv \sum_{k=0}^N c_k x^k e^{-x} \pmod{x^{N+1}}$ となる係数列 c_k を求める.
- c_k に $(k + b_0)(k + b_1) \cdots (k + b_{M-1})$ をかける.
- $\sum_{k=0}^N c_k x^k e^{-x} \pmod{x^{N+1}}$ を出力する.

A と c の間の変換は, e^{-x} や e^x をかけるだけです. $(k + b_0)(k + b_1) \cdots (k + b_{M-1})$ をかける部分については, $(x + b_0)(x + b_1) \cdots (x + b_{M-1})$ を多点評価すればよいです.

5. 関連問題

1. https://atcoder.jp/contests/arc133/tasks/arc133_f
2. <https://qoj.ac/contest/1917/problem/10095>
3. <https://yukicoder.me/problems/no/2583>
4. https://atcoder.jp/contests/abc272/tasks/abc272_h
5. <https://codeforces.com/contest/923/problem/E>

6. https://mofecoder.com/contests/itfpc2025/tasks/itfpc2025_n

▶ 問題へのコメント

6. まとめ

本講座では、形式的べき級数の微分を含むようなある種の dp を高速化するテクニックについて解説しました。

このテクニックは少数の難問で登場してきましたが、解法の関連性や、テクニックの応用範囲についてはあまり整理されてこなかったものだと思います。本講座でも応用範囲を明確にすることはしませんでした。いくつかの問題の解法を統一的に理解することはできました。

このテクニックは形式的べき級数の合成アルゴリズムが一般に高速化されたことによっても応用範囲が広がっています。今後出題例が増えたりさらに研究が進む可能性もあると思います。

7. 参考文献

特にありません。

トップページ：[AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など：[Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。 リアルタイムのオンラインコンテストで競い合うことや、 5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)